

Manus-Level Autonomous AI Software Engineering Platform — Master Specification

يجب بناء المنصة كمنظومة Autonomous AI Software Engineering متكاملة قادرة على تحويل طلب المستخدم باللغة الطبيعية إلى منتج برمجي حقيقي Production-ready، بدءًا من فهم الفكرة وتحليل المتطلبات، مرورًا بالتخطيط والتصميم والبرمجة والاختبار والتصحيح والأمن وتحسين الأداء، وانتهاءً بالنشر والمراقبة والصيانة والتطوير المستمر. يجب ألا تكون المنصة مجرد Chatbot أو AI Code Generator أو Website Builder، بل بيئة Agentic كاملة تمتلك عقلًا للتخطيط، ووكلاء متخصصين، وأدوات تنفيذ حقيقية، ومتصفّحًا حيًا، وTerminal، وFilesystem، وSandbox، وGit، وقاعدة معرفة، وذاكرة، ونظام تحقق وإصلاح ذاتي.

1. Design System

يجب بناء Design System احترافي وقابل للتوسع يحتوي على Design Tokens، Colors، Typography، Font، System، Spacing، Grid، Borders، Radius، Shadows، Elevation، Icons، Motion، Animation، Responsive Breakpoints، Dark Mode، Light Mode، Accessibility Rules، Visual States، Component Guidelines، Layout Rules، Visual Hierarchy، Themes، وResponsive Design.

يجب ألا يتم فرض شكل بصري واحد على جميع المشاريع. يجب أن يستطيع النظام إنشاء تصاميم مختلفة جذريًا حسب طبيعة المنتج والجمهور والهوية البصرية.

استخدام تقنيات مفتوحة المصدر مثل Tailwind CSS وshadcn/ui وRadix UI وLucide أو بدائل مناسبة.

2. UX System

يجب أن يحتوي النظام على UX Architecture كاملة تشمل User Flows، Information Architecture، Navigation، Search، Filtering، Sorting، Forms، Validation، Onboarding، Empty States، Loading States، Error States، Success States، Confirmation Flows، Notifications، Modals، Drawers، Mobile UX، Keyboard Navigation، Accessibility، Progressive Disclosure، Undo/Redo، Contextual Actions، Responsive Interaction Patterns، وBehavior.

يجب أن يتم تصميم التجربة بناءً على متطلبات المشروع وليس إعادة استخدام قالب ثابت.

3. Component System

إنشاء مكتبة Components قابلة لإعادة الاستخدام تشمل Buttons، Inputs، Selects، Dropdowns، Modals، Drawers، Tabs، Cards، Tables، Data Grids، Forms، Calendars، Command Menus، Tooltips، Popovers، Toasts، Alerts، Badges، Avatars، Navigation، Sidebar، Breadcrumbs، Pagination، Charts، File Upload، Rich Text Editor، Code Editor، Terminal، Logs، Dialogs، Wizards، Kanban، Timeline، Chat، AI Panels وغيرها.

كل Component يجب أن يمتلك جميع الحالات اللازمة: Default, Hover, Focus, Active, Disabled, Loading, Error, Success, Selected, Empty, Read-only, Responsive, Dark Mode.

4. UX Quality Engine

يجب أن يفحص النظام الواجهة من ناحية وضوح التسلسل البصري، سهولة الاستخدام، اتساق المكونات، جودة التنقل، عدد الخطوات المطلوبة لإكمال المهمة، جودة النماذج، وضوح الأخطاء، جودة حالات Loading و Empty و Success، وسهولة الاستخدام على الهاتف والكمبيوتر.

5. Frontend Engineering

دعم React و TypeScript و Vite أو Next.js حسب طبيعة المشروع.

يجب دعم Component Architecture, Routing, State Management, Server State, Client State, Forms, Validation, Data Fetching, Caching, Optimistic Updates, Error Boundaries, Loading UI, Responsive Design, Accessibility, SEO, Performance, Code Splitting, Lazy Loading, Image Optimization, Animations, Browser Compatibility, Progressive Enhancement.

6. Backend Engineering

إنشاء Backend Production-ready يشمل Authentication, Authorization, Sessions, APIs, Validation, Rate Limiting, File Uploads, Storage, Emails, Notifications, Payments, Webhooks, Background Jobs, Queues, Cron Jobs, Audit Logs, Error Handling, Logging, Monitoring, Security, External Integrations.

يجب فصل Business Logic عن Routes و Controllers و Database Access.

7. Database Engineering

استخدام PostgreSQL أو قاعدة مناسبة حسب المشروع.

يجب دعم Schema Design, Relationships, Primary Keys, Foreign Keys, Constraints, Indexes, Transactions, Migrations, Seeds, RLS, Query Optimization, Pagination, Search, Full-text Search, Backups, Data Validation, Data Integrity, Soft Delete, Audit Data, Database Security.

يجب تصميم قاعدة البيانات وعلاقاتها قبل بناء Business Logic المعتمد عليها.

8. API & Integration System

دعم Third-party APIs, REST APIs, tRPC, WebSockets, Webhooks, GraphQL عند الحاجة، و.

يجب توفير API Validation, Authentication, Authorization, Error Standards, Rate Limiting, Versioning, Documentation, Retries, Timeouts, Idempotency, Webhook Verification, Logging, Monitoring, Circuit Breaking عند الحاجة.

9. Authentication & Identity

دعم Email/Password, OAuth, Social Login, MFA, Sessions, Password Reset, Email Verification, Roles, Permissions, Organizations, Teams, Invitations, API Keys, Service Accounts, Session Revocation, Device Management, و Security Policies.

10. Authorization & RBAC

يجب أن يمتلك النظام Permission Engine يدعم:

Users → Organizations → Teams → Roles → Permissions → Resources → Actions.

ويجب أن يستطيع المشروع تحديد صلاحيات دقيقة على مستوى الموارد والعمليات.

11. Testing System

لا يعتبر المشروع مكتملاً حتى يتم اختباره.

يجب دعم Unit Tests, Component Tests, Integration Tests, API Tests, E2E Tests, Browser Tests, Regression Tests, Accessibility Tests, Visual Tests, Performance Tests, Security Tests, Database Tests, Smoke Tests, Contract Tests.

بعد كل تغيير مهم يجب إعادة تشغيل الاختبارات المناسبة.

12. Browser Automation & QA

يجب امتلاك Browser Agent حقيقي يستطيع فتح المواقع والتعامل معها كما يفعل المستخدم.

يجب دعمه لـ:

Navigation, Tabs, Click, Type, Scroll, Drag & Drop, Upload, Download, Login, Forms, JavaScript

Applications, Cookies, Sessions, Screenshots, Console, Network, Responsive Viewports, Multiple Pages, وComplex User Flows.

يجب عدم الاكتفاء بتحليل الكود؛ يجب اختبار المنتج الحقيقي داخل Browser.

13. Visual QA

يجب التقاط Screenshots وتحليل الواجهة بصريًا، واكتشاف:

Layout Errors, Overflow, Misalignment, Typography Problems, Spacing Problems, Broken Components, Responsive Problems, Contrast Problems, Missing Elements.

بعد الإصلاح يجب إعادة التصوير والتحليل حتى الوصول إلى مستوى الجودة المطلوب.

14. Security System

يجب أن تكون Security جزءًا من Architecture.

يجب حماية النظام من XSS, CSRF, SQL Injection, SSRF, Command Injection, Path Traversal, Authentication Attacks, Authorization Bugs, Insecure File Uploads, Secret Leakage, Dependency Vulnerabilities, Session Attacks, Rate Abuse, وغيرها من المخاطر المناسبة لنوع التطبيق.

يجب استخدام Secure Headers, Input Validation, Output Encoding, Encryption, Secure Cookies, Least Privilege, Secrets Management, Dependency Scanning, Audit Logs.

15. Sandbox & Code Execution

يجب تشغيل كود الـ Agent داخل بيئات معزولة.

كل Execution Environment يجب أن يحتوي على:

Filesystem Isolation, Process Isolation, Network Policies, CPU Limits, Memory Limits, Disk Limits, Timeouts, Package Management, Environment Variables, Container Isolation, Cleanup, وExecution Logs.

يجب فصل Execution Plane عن Control Plane.

16. Terminal Agent

يجب إعطاء الـ Agent Terminal حقيقياً قادراً على:

Install Dependencies, Run Commands, Run Tests, Build Projects, Start Servers, Inspect Logs, Run Scripts, Execute Migrations, Manage Git, Run Linters, Run Formatters, Run Security Scans.

يجب تطبيق Permission Policies على الأوامر الحساسة.

17. Filesystem Agent

يجب أن يستطيع الـ Agent:

Create, Read, Edit, Rename, Move, Delete, Search, Diff, Compare, Analyze, Refactor, Organize.

ويجب أن تكون كل التغييرات قابلة للتتبع والاسترجاع.

18. Git System

يجب أن يكون Git جزءاً أساسياً.

دعم, Branches, Commits, Diff, Revert, Rollback, Tags, Pull Requests, Code Review, Change History, Safe Checkpoints, Release Management.

كل مجموعة تغييرات مهمة يجب أن تكون قابلة للرجوع.

19. Codebase Intelligence

يجب أن يفهم الـ Agent المشروع ككل.

يجب فهرسة:

Files, Symbols, Imports, Exports, Components, Routes, APIs, Database Relations, Tests, Configuration, Dependencies, Environment Variables, Services.

ويجب أن يستطيع معرفة تأثير تعديل ملف على بقية المشروع.

20. Dependency Intelligence

يجب مراقبة Dependencies واكتشاف:

Outdated Packages, Vulnerabilities, Breaking Changes, Conflicts, Peer Dependency Problems, License Issues, Compatibility Problems.

21. Architecture Engine

قبل كتابة الكود يجب تحليل المشروع وتحديد:

Frontend Architecture, Backend Architecture, Database Architecture, API Architecture, Authentication, State Management, Storage, Infrastructure, Integrations, Testing Strategy, Security Model, Deployment Strategy.

يجب أن تتناسب Architecture مع حجم المشروع.

22. Requirements Engineering

يجب تحويل Prompt المستخدم إلى:

Requirements → User Stories → Acceptance Criteria → Functional Requirements → Non-functional Requirements → Constraints → Dependencies → Risks → Technical Plan.

إذا كان الطلب غامضًا، يجب اكتشاف النقاط التي تؤثر فعليًا على التنفيذ وطلب التوضيح فقط عند الضرورة.

23. Planning Engine

يجب إنشاء خطة تنفيذية قابلة للتنفيذ وليس مجرد قائمة عامة.

يجب إنشاء Task Graph يحتوي على Dependencies و Parallel Tasks و Critical Path و Milestones و Quality Gates.

24. Agent Orchestrator

يجب وجود Orchestrator مركزي يدير الوكلاء والمهام.

الحالات:

IDLE → ANALYZING → PLANNING → WAITING_FOR_INPUT → EXECUTING → TESTING → FIXING → VERIFYING → AWAITING_APPROVAL → DEPLOYING → MONITORING → COMPLETED / FAILED /

PAUSED / CANCELLED.

25. Specialized Agents

يجب دعم Agents متخصصة مثل:

Requirements Agent
Product Agent
Planner Agent
Architect Agent
UI/UX Agent
Frontend Agent
Backend Agent
Database Agent
Integration Agent
Testing Agent
Browser QA Agent
Security Agent
Performance Agent
SEO Agent
Accessibility Agent
Code Review Agent
Debug Agent
DevOps Agent
Deployment Agent
Monitoring Agent
Maintenance Agent

يجب ألا تعمل الوكلاء بشكل عشوائي، بل ضمن Orchestration واضحة.

26. Agent Communication

يجب أن يمتلك كل Agent بروتوكولاً موحداً للتواصل يتضمن:

Task, Context, Inputs, Outputs, Artifacts, Dependencies, Errors, Status, Confidence, Handoff.

27. Parallel Execution

يجب تنفيذ المهام المستقلة بالتوازي عند الإمكان مع إدارة تعارضات الملفات والموارد.

يجب ألا يسمح لعدة Agents بتعديل نفس المورد بطريقة تؤدي إلى Corruption.

28. Planning & Replanning

الـ Agent لا يجب أن يلتزم بالخطة الأولى بشكل أعمى.

يجب أن يعمل:

Plan → Execute → Observe → Evaluate → Replan → Continue.

إذا ظهرت مشكلة أو تغيرت المتطلبات، يعيد بناء الجزء المتأثر من الخطة.

29. Context Engineering

يجب إدارة Context بذكاء.

يشمل ذلك:

Context Selection, Code Retrieval, Relevant Files, Dependency Context, Conversation Context, Project Context, Summarization, Compression, Token Budgeting, Context Prioritization, و Task-specific Context.

لا يجب إرسال المشروع كاملاً للنموذج في كل خطوة.

30. Memory System

يجب فصل:

Short-Term Memory

Project Memory

User Preferences

Task Memory

Technical Decisions

Error Memory

Tool Memory

ويجب تخزين القرارات المهمة والنتائج التي يحتاجها المشروع مستقبلاً.

31. Knowledge System

يجب بناء Knowledge Base تحتوي على:

Framework Documentation, API Documentation, Coding Patterns, Architecture Patterns, UI Patterns, Security Rules, Testing Patterns, Database Patterns, Deployment Patterns, Common Errors, Solutions, Code Examples, Templates, Best Practices.

يجب استخدام Retrieval لتوفير المعرفة المناسبة للمهمة.

32. Tool System

يجب أن يمتلك الـ Agent أدوات حقيقية:

Browser, Terminal, Filesystem, Git, Database, API Client, Search, Documentation, Screenshot, Test Runner, Build System, Docker, SSH, Deployment, Monitoring, Storage وغيرها.

33. Tool Discovery

لا يجب إعطاء كل Agent جميع الأدوات في كل مهمة.

يجب أن يستطيع النظام اكتشاف الأدوات المطلوبة ديناميكياً بناءً على المهمة.

34. Tool Permission System

كل Tool يجب أن يدعم:

ALLOW
DENY
ASK USER

ويجب تطبيق Least Privilege.

العمليات الحساسة مثل حذف البيانات أو النشر إلى Production أو إرسال معلومات خارجية يجب أن تخضع لسياسات Approval.

35. Secrets Vault

يجب حماية:

API Keys, OAuth Tokens, SSH Keys, Database Credentials, Environment Secrets, Service Credentials.

يجب ألا يتم كشف Secrets للنموذج إلا عند الضرورة القصوى وبصلاحيات محدودة.

36. OAuth & Integrations Platform

يجب دعم تكاملات خارجية مثل:

GitHub, Google, Gmail, Slack, Discord, Notion, Stripe, Supabase, Cloud Providers, Storage Providers, CRM, Calendar, WhatsApp وغيرها.

يجب أن يكون كل Integration قابلاً للفصل والاستبدال.

37. Computer-Use System

يجب أن يستطيع Agent التعامل مع الكمبيوتر والواجهات الرسومية عندما تتطلب المهمة ذلك.

يشمل:

Mouse, Keyboard, Browser, Windows, Applications, Files, Screenshots, Drag & Drop, Clipboard, Downloads, Uploads.

38. Browser Profiles

يجب دعم Browser Sessions و Profiles منفصلة مع Cookies و Storage و Authentication States، مع عزل جلسات المستخدمين والمشاريع.

39. Artifact System

كل مهمة يجب أن تنتج Artifacts قابلة للتتبع مثل:

Code, Files, Screenshots, Reports, Tests, Logs, Database Schemas, Deployments, Documents, Generated Assets.

40. Checkpoint System

يجب إنشاء Checkpoints خلال العمل.

إذا حدث Crash أو Timeout أو انقطاع API يجب أن يستطيع النظام استئناف المهمة من آخر حالة صحيحة.

41. Resume System

يجب حفظ:

Plan, Task State, Files, Artifacts, Context, Results, Errors, Agent State, Tool Results.

وبالتالي:

Pause → Resume

بدون إعادة تنفيذ المشروع من الصفر.

42. Failure Recovery

يجب التعامل مع:

API Failure, Model Failure, Network Failure, Timeout, Browser Crash, Container Crash, Server Failure, Database Failure, Deployment Failure, Rate Limits.

مع .Retry, Backoff, Fallback, Recovery, Rollback

43. Self-Healing

يجب أن يعمل النظام:

Detect → Diagnose → Fix → Retest → Verify.

عند ظهور خطأ يجب تحليل Root Cause وليس فقط تعديل السطر الذي ظهر فيه الخطأ.

يجب تحديد Maximum Attempts ومنع Infinite Loops.

44. Regression Protection

بعد كل إصلاح يجب تشغيل الاختبارات المتأثرة والتأكد من أن الإصلاح لم يكسر وظائف موجودة.

45. Code Review Engine

بعد التنفيذ يجب تحليل:

Architecture, Readability, Maintainability, Security, Performance, Error Handling, Type Safety, Duplication, Complexity, Testing Coverage, Dependency Usage.

46. Quality Gates

لا تعتبر المهمة مكتملة لمجرد أن Agent أعلن نجاحها.

يجب اجتياز:

Requirements Complete

Architecture Valid

Type Check Passed

Lint Passed

Tests Passed

Build Passed

Security Passed

Accessibility Passed

Performance Acceptable

Browser QA Passed

Visual QA Passed

Deployment Healthy

47. Performance System

يجب تحسين:

Frontend, Backend, Database, APIs, Images, Fonts, JavaScript, Network, Caching, CDN.

يجب قياس Core Web Vitals, Bundle Size, API Latency, Database Queries, Server Response Time.

48. SEO System

دعم:

Metadata, Dynamic Titles, Descriptions, Canonical URLs, Sitemap, Robots, Structured Data, Open Graph, Twitter Cards, Semantic HTML, Internal Linking, Image Alt Text, SSR/SSG عند الحاجة.

49. Accessibility System

دعم WCAG المناسب للمشروع.

يشمل:

Keyboard Navigation, Focus Management, Screen Readers, Semantic HTML, ARIA, Contrast, Accessible Forms, Error Messages, Reduced Motion, Touch Targets.

50. DevOps System

Pipeline:

Git → Branch → Lint → Type Check → Tests → Security Scan → Build → Docker → Deploy → Health Check → Smoke Test → Monitoring.

51. Deployment System

يجب دعم:

SSH, Docker, Nginx, SSL, Domains, Environment Variables, Database Configuration, Containers, Health Checks, Smoke Tests, Rollback, Deployment Logs.

52. Production Verification

بعد النشر يجب اختبار:

Application Availability, Database, Authentication, Core User Flows, APIs, Browser, Responsive Design, Security, Performance, Logs, External Integrations.

53. Monitoring & Observability

يجب مراقبة:

Application Logs, Errors, Performance, Server Metrics, Database Metrics, Uptime, API Latency, Failed Requests, Deployment Status, Resource Usage, Alerts.

54. Agent Observability

يجب مراقبة الـ AI نفسه:

Model, Latency, Tokens, Cost, Tool Calls, Failed Calls, Retries, Task Duration, Success Rate, Model Performance.

55. Cost & Model Router

يجب اختيار النموذج المناسب لكل مهمة.

للتحليل Vision Model, للمهام البسيطة Fast Model, للبرمجة Coding Model, للمهام المعقدة Strong Model, للبصري Cheap Model, للمهام منخفضة التعقيد.

يجب دعم Fallback Models ومراقبة التكلفة.

56. Evaluation System

يجب امتلاك Benchmark داخلي يختبر قدرة النظام على تنفيذ مجموعة كبيرة من المهام.

يجب قياس:

Task Success, Code Quality, UI Quality, Security, Performance, Reliability, Cost, Speed.

57. Continuous Improvement

بعد كل مهمة:

Result → Evaluation → Feedback → Knowledge Update → System Improvement.

يجب استخدام النتائج لتحسين النظام، وليس فقط تخزينها.

58. Multi-Project System

يجب أن يستطيع المستخدم إدارة مشاريع متعددة مع فصل:

Files, Memory, Context, Secrets, Deployments, Databases, Integrations, Environments.

59. Environments

دعم:

Development → Staging → Production.

مع Environment Variables و Secrets و Databases و Deployments منفصلة.

60. Rollback

كل Deployment يجب أن يكون قابلاً للعودة إلى نسخة مستقرة سابقة.

61. Backup & Disaster Recovery

يجب توفير:

Database Backups, Project Backups, Configuration Backups, Deployment History, Recovery Procedures, Disaster Recovery.

62. Scalability

يجب تصميم المنصة بحيث يمكن الانتقال من مستخدم واحد إلى عدد كبير من المستخدمين.

يشمل ذلك:

Queues, Workers, Caching, Horizontal Scaling, Database Scaling, CDN, Object Storage, Rate Limiting, Resource Isolation.

63. Multi-Tenant Architecture

إذا أصبحت المنصة SaaS يجب دعم:

Organizations, Users, Teams, Projects, Roles, Permissions, Billing, Usage, Quotas, Isolation.

64. Usage & Billing

دعم:

Subscriptions, Credits, Tokens, Model Costs, Compute Usage, Storage, Agent Minutes, Quotas, Invoices.

65. Audit & Forensics

يجب تسجيل:

من قام بالعملية، ماذا حدث، متى حدث، أي Agent استخدم، أي Tool استخدم، ما الملفات المتأثرة، ما APIs المستخدمة، وما النتيجة.

66. Safety Engine

يجب وجود طبقة تمنع العمليات الخطرة غير المصرح بها.

تشمل:

Policy Enforcement, Permission Checks, Resource Limits, Sandbox, Secret Protection, User Approval, Network Restrictions, File Restrictions, Production Protection.

67. Project Understanding

يجب أن يستطيع النظام بناء Model داخلي للمشروع يربط:

Requirements → Pages → Components → APIs → Database → Services → Tests → Deployment.

وبذلك يستطيع فهم تأثير أي تغيير.

68. Product Intelligence

يجب أن يستطيع Agent اقتراح تحسينات منطقية مثل:

Missing Features, UX Improvements, Security Risks, Performance Problems, SEO Improvements, Accessibility Issues, Testing Gaps.

لكن لا ينفذ تغييرات جوهرية تلقائيًا بدون الصلاحية المناسبة.

69. Natural Language Interface

يجب أن يستطيع المستخدم التعامل مع النظام باللغة الطبيعية:

"أضف نظام دفع."

"غيّر التصميم."

"أصلح الخطأ."

"انشر المشروع."

"أضف لوحة تحكم."

"اجعل الموقع متوافقًا مع الهاتف."

"راجع الأمان."

ويجب تحويل الطلب تلقائيًا إلى Tasks قابلة للتنفيذ.

70. Interactive Development

يجب أن يستطيع المستخدم رؤية:

Chat, Agent Activity, Task Progress, Files, Code Diff, Terminal, Browser Preview, Screenshots, Logs, Tests, Deployment Status.

ويجب أن يكون كل شيء متزامنًا مع حالة المشروع الحقيقية.

71. Real-Time Activity System

عرض الأحداث مباشرة:

Agent Started

Planning

Reading Files

Writing Code
Running Command
Running Test
Browser Opened
Error Detected
Fixing
Deployment Started
Deployment Completed

بدون Fake Activity.

72. Human-in-the-Loop

عند الحاجة إلى قرار بشري يجب عرض:

Action, Reason, Risk, Files, Cost, Permissions, Expected Result.

ثم:

Approve / Reject / Modify.

73. Project Collaboration

إذا أصبحت المنصة متعددة المستخدمين, يجب دعم:

Teams, Roles, Comments, Mentions, Activity, Shared Projects, Permissions, Reviews.

74. Professional Project Templates

يجب توفير Templates مرجعية للمشاريع الشائعة:

SaaS, E-commerce, Marketplace, CRM, ERP, Booking, Hotel, Restaurant, Learning, Social, Portfolio, Dashboard, AI Apps, Admin Systems, Internal Tools, API Platforms.

لكن يجب أن تكون Templates نقطة بداية وليس قيدًا على النظام.

75. Final Autonomous Development Loop

يجب أن تكون دورة العمل الأساسية:

USER IDEA



REQUIREMENTS



PRODUCT SPECIFICATION



UX



ARCHITECTURE



TASK GRAPH



IMPLEMENTATION



RUN



OBSERVE



TEST



DETECT



DIAGNOSE



FIX



RETEST



SECURITY AUDIT



PERFORMANCE AUDIT

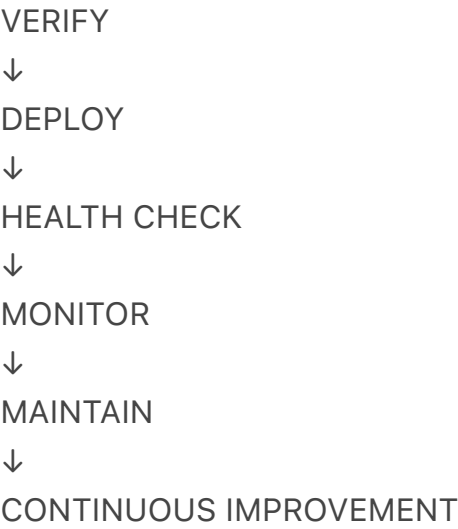


VISUAL QA



CODE REVIEW





76. Core Architecture

يجب تقسيم المنصة إلى طبقات واضحة:

AI Control Plane

Models, Orchestrator, Planning, Memory, Context, Policies, Agent Management.

Agent Runtime

Tasks, Agents, State, Handoffs, Parallel Execution, Checkpoints, Recovery.

Tool Plane

Browser, Terminal, Filesystem, Git, Database, APIs, Search, Documentation, Deployment.

Execution Plane

Sandbox, Containers, Processes, Network, Files, Resources.

Engineering Plane

Frontend, Backend, Database, Testing, Security, Performance, DevOps.

Integration Plane

OAuth, APIs, External Services, Cloud, GitHub, Communication, Payments.

Observability Plane

Logs, Metrics, Traces, Agent Monitoring, Error Tracking, Evaluation.

User Experience Plane

Chat, Dashboard, Project Workspace, Browser Preview, Code Editor, Terminal, Activity, Approval UI.

77. Core Principle

الهدف ليس بناء نموذج ذكاء اصطناعي يكتب كودًا فقط.

الهدف هو بناء:

Autonomous AI Software Engineering Platform

تستطيع فهم فكرة المستخدم، التفكير في Architecture، التخطيط، استخدام الأدوات، إنشاء الملفات، تشغيل Terminal، استخدام Browser، بناء قاعدة البيانات، كتابة Frontend و Backend، ربط APIs، اختبار التطبيق، تحليل الأخطاء، إصلاحها، مراجعة الكود، فحص الأمان، تحسين الأداء، نشر المشروع، مراقبته، واستكمال تطويره لاحقًا.

يجب أن تكون المنصة قادرة على إنشاء مشاريع مختلفة جذريًا من حيث التصميم والوظائف والـ Architecture، وليس إعادة استخدام قالب واحد وتغيير النصوص والألوان.

ويجب ألا تعلن المنصة أن المهمة انتهت بناءً على كلام الـ AI؛ بل بناءً على أدلة قابلة للتحقق من الاختبارات، Browser QA، Production Verification، Visual QA، Build، Security، Performance، Deployment Health.

القاعدة النهائية:

PLAN → BUILD → RUN → OBSERVE → TEST → DETECT → DIAGNOSE → FIX → RETEST → VERIFY → DEPLOY → MONITOR → MAINTAIN → IMPROVE

ويجب أن تكون جميع العمليات قابلة للتتبع، قابلة للاستئناف، قابلة للرجوع، آمنة، معزولة، وقابلة للتوسع.

النتيجة المستهدفة: منصة Autonomous AI Software Engineer قادرة على بناء وتشغيل وصيانة مشاريع Production حقيقية بمستوى احترافي قريب من فئة Manus/Replit، مع الحفاظ على مرونة كاملة في نوع المشروع وتصميمه ومعماريته.